

🔥 Full Stack Development – React, NodeJS, MongoDB & TypeScript Cheatsheet

ReactJS

ReactJS is a JavaScript library for building user interfaces, primarily for single-page applications. It uses a component-based architecture and a virtual DOM for efficient updates.

Core Concepts

- Components:** Reusable, isolated pieces of UI.
- JSX:** JavaScript XML, a syntax extension for React, allowing HTML-like code in JS.
- Virtual DOM:** A light-weight copy of the actual DOM for faster updates.
- Props:** Data passed from parent to child components (read-only).
- State:** Data managed within a component, can change over time.
- Hooks:** Functions that let you “hook into” React state and lifecycle features from function components.

Important Syntax & Code Patterns

1. Functional Component

```
import React from 'react';

function MyComponent(props) {
  return (
    <div>
      <h1>Hello, {props.name}!</h1>
    </div>
  );
}

export default MyComponent;
```

2. Using State (useState Hook)

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // [currentState, updaterFunction]

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

3. Using Effects (useEffect Hook)

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prevSeconds => prevSeconds + 1);
    }, 1000);

    // Cleanup function
    return () => clearInterval(interval);
  }, []); // Empty dependency array means it runs once on mount and cleans up

  return <p>Seconds: {seconds}</p>;
}
```

4. Conditional Rendering

```
function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in.</h1>}
    </div>
  );
}
```

5. List Rendering with Keys

```
function ItemList({ items }) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

Commonly Used Methods / Hooks

- useState(): Manages state in functional components.
- useEffect(): Handles side effects (data fetching, subscriptions, manual DOM changes).
- useContext(): Consumes values from React Context.
- useRef(): Creates mutable ref objects that persist across renders.
- useReducer(): Alternative to useState for complex state logic.
- useCallback(): Memoizes functions to prevent unnecessary re-renders.
- useMemo(): Memoizes values to prevent unnecessary re-calculations.

Example Use Cases

- Component:** <Button /> component for reusable buttons.
- State:** Managing user input in a form field.
- Props:** Passing a user's name to a <UserProfile /> component.
- useEffect:** Fetching data from an API when a component mounts.
- React Router:** Navigating between /home and /about pages.

Quick Difference: Props vs State

Feature	Props	State
Definition	Arguments passed into components.	Data managed within a component.
Mutability	Immutable (read-only).	Mutable (can be changed).
Source	Passed from parent component.	Managed by the component itself.
Purpose	Component communication, configuration.	Internal data management, dynamic updates.

Viva / Interview Questions

- Q:** Explain the concept of Virtual DOM in React. Why is it used?
A: Virtual DOM is a lightweight copy of the actual DOM. React uses it to improve performance by first updating the Virtual DOM, then efficiently calculating the minimal changes needed to update the real DOM, reducing direct manipulation.
- Q:** What are React Hooks, and what problem do they solve?
A: Hooks are functions that let you use state and other React features in functional components. They solve the problems of class components: complex lifecycle methods, difficulty in reusing stateful logic, and confusing this keyword.
- Q:** When should you use useState versus useReducer?
A: Use useState for simpler state logic, typically when state updates are straightforward. Use useReducer for more complex state logic involving multiple sub-values or when the next state depends on the previous one.

NodeJS & ExpressJS

NodeJS: A JavaScript runtime built on Chrome's V8 JavaScript engine, allowing server-side execution of JavaScript.

ExpressJS: A fast, unopinionated, minimalist web framework for Node.js, simplifying server-side development.

Core Concepts

- Event-Driven, Non-blocking I/O:** Node.js handles concurrent operations efficiently.
- Modules:** Reusable code units (e.g., http, fs, path).
- NPM (Node Package Manager):** Package manager for Node.js, managing dependencies.
- Middleware:** Functions that have access to the request and response objects in Express.
- Routing:** Defining how an application responds to client requests to specific endpoints.

Important Syntax & Code Patterns

1. Basic Express Server

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.listen(PORT, () => {
  console.log('Server running on http://localhost:${PORT}');
});
```

2. Middleware Example (Logger)

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} at ${new Date()}`);
  next(); // Pass control to the next middleware/route handler
});

app.get('/users', (req, res) => {
  res.json([{ id: 1, name: 'Alice' }]);
});
```

3. Handling Route Parameters

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`Fetching user with ID: ${userId}`);
});
```

4. Handling POST Requests & Form Data (with body-parser or built-in Express)

```
app.use(express.urlencoded({ extended: true })); // For parsing application/x-www-form-urlencoded
app.use(express.json()); // For parsing application/json
```

```
app.post('/submit-form', (req, res) => {
  const { username, password } = req.body;
  res.send(`Received: Username - ${username}, Password - ${password}`);
});
```

5. Using a Templating Engine (EJS)

```
// Install EJS: npm install ejs
app.set('view engine', 'ejs');
app.set('views', './views'); // Specify views directory

app.get('/profile', (req, res) => {
  const user = { name: 'John Doe', email: 'john@example.com' };
  res.render('profile', { user: user }); // Renders views/profile.ejs
});
```

Commonly Used Methods / Functions

- app.listen(port, callback): Starts the server.
- app.get(path, callback), app.post(path, callback), etc.: Defines routes for HTTP methods.
- app.use(middleware): Mounts middleware functions.
- res.send(): Sends a response of various types.
- res.json(): Sends a JSON response.
- res.render(view, data): Renders a view template.
- req.params: Object containing route parameters.
- req.query: Object containing query string parameters.
- req.body: Object containing parsed request body data (requires middleware).

Example Use Cases

- NodeJS:** Building a REST API, command-line tools, real-time chat applications.
- ExpressJS:** Creating endpoints for a user management system (/users, /users/:id).
- Middleware:** Logging all incoming requests, authenticating users before accessing routes.
- Templating:** Displaying dynamic content on a server-rendered web page.
- Session Management:** Storing user login status in a cookie.

Quick Difference: GET vs POST

Feature	GET	POST
Purpose	Retrieve data from server.	Submit data to be processed to a specified resource.
Parameters	Sent in URL query string.	Sent in request body.
Visibility	Visible in URL, browser history.	Not visible in URL.
Idempotent	Yes (multiple identical requests have same effect).	No (multiple identical requests may have different effects).
Security	Less secure for sensitive data.	More secure for sensitive data.

Viva / Interview Questions

- Q:** What is the event loop in Node.js?
A: The event loop is a core component of Node.js's non-blocking I/O model. It continuously checks the call stack for tasks to execute and the callback queue for events to process, allowing Node.js to handle many concurrent operations without blocking the main thread.
- Q:** Explain the role of middleware in Express.js. Give an example.
A: Middleware functions in Express.js are functions that execute in the middle of a request-response cycle. They can perform tasks like logging, authentication, data parsing, or error handling. An example is express.json() to parse JSON request bodies before route handlers.
- Q:** How do you handle user authentication and session management in Express.js?
A: User authentication typically involves checking credentials (e.g., username/password) against a database. Session management often uses libraries like express-session or JWTs (JSON Web Tokens). express-session creates a session object on the server and sets a cookie with a session ID on the client. JWTs are self-contained tokens sent with each request for stateless authentication.

MongoDB

MongoDB is a NoSQL, document-oriented database program. It stores data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.

Core Concepts

- Document:** A record in MongoDB (JSON-like BSON format).
- Collection:** A group of documents (analogous to a table in relational databases).
- Database:** A container for collections.
- Schema-less:** Documents in a collection don't need to have the same fields or structure.
- Scalability:** Designed for horizontal scaling using sharding.

Important Syntax & Code Patterns (using mongodb driver or Mongoose)

1. Connecting to MongoDB (Node.js with mongodb driver)

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri);

async function connectDB() {
  try {
    await client.connect();
    console.log('Connected to MongoDB!');
    return client.db('mydatabase'); // Returns the database object
  } catch (error) {
    console.error('MongoDB connection error:', error);
  }
}
```

2. Insert Document(s)

```
async function insertData(db) {
  const collection = db.collection('users');
  // Insert one
  await collection.insertOne({ name: 'Alice', age: 30 });
  // Insert many
  await collection.insertMany([
    { name: 'Bob', age: 25 },
    { name: 'Charlie', age: 35 }
  ]);
  console.log('Documents inserted.');
```

3. Find Documents

```
async function findData(db) {
  const collection = db.collection('users');
  // Find all
  const allUsers = await collection.find({}).toArray();
  console.log('All users:', allUsers);
  // Find with query
  const youngUsers = await collection.find({ age: { $lt: 30 } }).toArray();
  console.log('Users younger than 30:', youngUsers);
}
```

4. Update Documents

```
async function updateData(db) {
  const collection = db.collection('users');
  // Update one
  await collection.updateOne(
    { name: 'Alice' },
    { $set: { age: 31, status: 'active' } }
  );
  // Update many
  await collection.updateMany(
    { age: { $lt: 30 } },
    { $inc: { age: 1 } } // Increment age by 1
  );
  console.log('Documents updated.');
```

5. Delete Documents

```
async function deleteData(db) {
  const collection = db.collection('users');
  // Delete one
  await collection.deleteOne({ name: 'Bob' });
  // Delete many
  await collection.deleteMany({ age: { $gt: 30 } });
  console.log('Documents deleted.');
```

Commonly Used Methods / Functions

- db.collection('name'): Accesses a specific collection.
- collection.insertOne(doc): Inserts a single document.
- collection.insertMany([docs]): Inserts multiple documents.
- collection.find(query): Returns a cursor for documents matching the query.
- collection.findOne(query): Returns a single document matching the query.
- collection.updateOne(filter, update): Updates a single document.
- collection.updateMany(filter, update): Updates multiple documents.
- collection.deleteOne(filter): Deletes a single document.
- collection.deleteMany(filter): Deletes multiple documents.
- .toArray(): Converts a cursor to an array of documents.

Example Use Cases

- Insert:** Adding a new user registration record.
- Find:** Retrieving all products in an e-commerce catalog.
- Update:** Changing a user's address or updating a product's price.
- Delete:** Removing a user account or a cancelled order.

Viva / Interview Questions

- Q:** What is the main difference between SQL and NoSQL databases, specifically MongoDB?
A: SQL databases are relational, schema-based, and use structured query language, ideal for complex transactions and fixed data structures. NoSQL databases like MongoDB are non-relational, schema-less (document-oriented), and use flexible JSON-like documents, better for large volumes of unstructured data and horizontal scalability.
- Q:** Explain what a "document" and a "collection" are in MongoDB.
A: A **document** is the basic unit of data in MongoDB, stored in BSON (Binary JSON) format, similar to a row in a relational table but with a flexible schema. A **collection** is a group of documents, analogous to a table, but all documents within a collection do not need to have the same structure.
- Q:** How does MongoDB handle relationships between data, given it's a NoSQL database?
A: MongoDB handles relationships primarily through embedding (nesting related data within a single document) or referencing (storing IDs of related documents in another document). The choice depends on data access patterns and whether the related data is frequently accessed together or independently.

TypeScript

TypeScript is a superset of JavaScript that adds static typing to the language. It compiles to plain JavaScript and helps catch errors early during development.

Core Concepts

- Static Typing:** Types are checked at compile-time, not runtime.
- Type Annotations:** Explicitly defining types for variables, parameters, return values.
- Interfaces:** Define contracts for objects, enforcing structure.
- Enums:** A way to define a set of named constants.
- Union Types:** Allows a variable to hold one of several types (e.g., string | number).
- Generics:** Write code that works with a variety of types, providing flexibility and type safety.

Important Syntax & Code Patterns

1. Simple Types & Type Annotations

```
let userName: string = "Alice";
let userAge: number = 30;
let isActive: boolean = true;
let data: any = "Can be anything"; // Avoid 'any' if possible
```

2. Array Types

```
let names: string[] = ["Alice", "Bob"];
let ages: Array<number> = [25, 30, 35];
```

3. Function Parameters & Return Types

```
function add(a: number, b: number): number {
  return a + b;
}

const greet = (name: string): string => {
  return `Hello, ${name}!`;
};

function logMessage(message: string | number): void { // void for no return val
  console.log(message);
}
```

4. Interfaces (Defining Object Shapes)

```
interface User {
  id: number;
  name: string;
  email?: string; // Optional property
  age: number;
  greet(): string;
}

let user1: User = {
  id: 1,
  name: "Charlie",
  age: 40,
  greet: function() { return `Hi, I'm ${this.name}`; }
};
```

5. Union Types & Type Aliases

```
type ID = string | number; // Type alias

function printID(id: ID) {
  console.log(`My ID is: ${id}`);
}

printID("abc-123");
printID(12345);
```

6. Generics

```
function identity<T>(arg: T): T {
  return arg;
}

let output1 = identity<string>("myString"); // type of output1 is string
let output2 = identity<number>(100); // type of output2 is number
```

Commonly Used Types & Keywords

- Primitive Types:** number, string, boolean, null, undefined, symbol, bigint.
- Special Types:** any, unknown, void, never, object.
- Keywords:** interface, type, enum, class, public, private, protected.
- Type Assertions:** <Type>value or value as Type.

Example Use Cases

- Function Parameters:** Ensuring a function always receives a number for calculations.
- Interfaces:** Defining the expected structure for data received from an API.
- Union Types:** A variable that can accept either a string ID or a number ID.
- Generics:** Creating a reusable list component that can hold items of any type while maintaining type safety.

Viva / Interview Questions

- Q:** What is the primary benefit of using TypeScript over JavaScript?
A: The primary benefit is static typing, which allows developers to catch type-related errors during development (compile-time) rather than at runtime. This leads to more robust, maintainable code, better tooling, and improved developer experience through autocompletion and early error detection.
- Q:** Explain the difference between an interface and a type in TypeScript.
A: Both interface and type define the shape of objects. However, interface can be implemented by classes, merged (re-opened) for declaration merging, and is generally preferred for defining object shapes. type aliases can define primitive, union, intersection, and tuple types, and can also define object shapes but cannot be merged or implemented by classes.
- Q:** When would you use the any type, and why is it generally discouraged?
A: The any type can be used when you don't know the type of a variable, are migrating a JavaScript project, or dealing with third-party libraries without type definitions. It's discouraged because it bypasses TypeScript's type checking, effectively turning off static analysis for that variable, which defeats the purpose of using TypeScript and can lead to runtime errors.